

Fine-Tuning LLMs for Bug Detection in Quantum Software

Md Saidur Rahman

Department of Computer Science, Louisiana State University

mrahm65@lsu.edu

CSC 7644: Applied LLM Development | May 9, 2026

Code: <https://github.com/saidur-rahman/csc7644-final-project-rahman>

Abstract

This project studies whether code-aware language-model pre-training improves binary classification of Qiskit bug reports as either classical or quantum-specific. Quantum software bugs are difficult to triage because defects such as incorrect qubit mappings, gate misuse, and measurement-ordering errors require domain knowledge that traditional debugging tools do not encode. I implemented a controlled comparison of three 125M-parameter RoBERTa-architecture encoders: `roberta-base`, `microsoft/codebert-base`, and `microsoft/graphcodebert-base`. Each model was fine-tuned with the same preprocessing pipeline, class-weighted cross-entropy, minority oversampling, and macro-F1 early stopping. Evaluation used stratified 5-fold cross-validation repeated across five random seeds, producing 25 fold-runs per backbone. All three backbones exceeded the original 70% accuracy target, with CodeBERT achieving the highest mean accuracy and macro-F1, while GraphCodeBERT achieved the highest mean ROC-AUC. However, pairwise statistical tests showed no significant differences among the three models. The main lesson is that, at this dataset scale, data construction and imbalance handling mattered more than the specific pre-training corpus.

1 Introduction and Background

Quantum computing is moving from theoretical research into practical software engineering. Frameworks such as IBM’s Qiskit allow developers to write, simulate, and execute quantum programs, but quantum software introduces failure modes that do not appear in ordinary classical programs. Incorrect qubit mappings, gate misuse, measurement-ordering errors, and entanglement-related mistakes can change program behavior while leaving the code syntactically valid. These faults are difficult for traditional debugging tools because such tools generally lack quantum semantic awareness [7].

Today, quantum bug triage is largely manual. Developers inspect issue descriptions, code snippets, stack traces, and domain-specific terminology to decide whether a report reflects an ordinary software problem or a quantum-specific defect. This process is slow, requires expertise, and becomes harder to scale as quantum software ecosystems grow. An automated triage model would not replace quantum developers, but it could prioritize reports, route them to the correct maintainers, and reduce the cost of early diagnosis.

An LLM-based solution is appropriate because bug reports combine natural language, library names, code fragments, and domain-specific vocabulary. Pre-trained encoders such as CodeBERT [3] and GraphCodeBERT [4] have already shown that language models can represent program-related text effectively. This project therefore fine-tunes and compares three encoder backbones for classical-versus-quantum bug discrimination. The goal is both practical and methodological: to build a reproducible classifier and to test whether code-aware pre-training provides a measurable advantage over a natural-language-only encoder at the available data scale.

2 System Design and Implementation

2.1 Dataset

The labeled corpus consists of Qiskit bug entries drawn from Bugs-QCP and related quantum bug benchmark resources [1, 7]. Samples were filtered to retain entries explicitly labeled as either classical or quantum. The resulting dataset is moderately imbalanced, with classical bugs forming the majority class. Figure 1 shows the final class distribution used for model training and evaluation.

Missing figure file: `fig10dataset_distribution.png`
Alternative also missing: `fig_dataset_distribution.png`

Figure 1: Class distribution of labeled Qiskit bug samples after filtering.

Each bug entry was converted into a canonical input string by concatenating its name, description, and available example-code fields using newline separators. Empty fields were dropped. The resulting text was tokenized using the active backbone’s tokenizer and truncated to 256 tokens. This preprocessing choice ensured that every model received the same information, so performance differences could be attributed to the backbone rather than to different input construction.

2.2 Model Selection and Comparative Design

The experiment compares three models with the same RoBERTa-style encoder architecture and approximately the same parameter count. Holding architecture and capacity fixed isolates the main experimental variable: the pre-training corpus. Table 1 summarizes the three backbones.

Table 1: Encoder backbones compared under the same training and evaluation protocol.

Backbone	Pre-training Corpus	Purpose in Study
roberta-base [6]	Natural language only	Control model without code-aware pre-training.
microsoft/codebert-base [3]	Code and natural language	Tests whether code-aware bimodal pre-training improves classification.
microsoft/graphcodebert-base [4]	Code, natural language, and data-flow information	Tests whether structure-aware pre-training provides additional benefit.

2.3 Pipeline and Architecture

Figure 2 shows the full workflow. Raw bug entries first pass through filtering and canonical text construction. The same processed text is then used for all three backbones. For each model, stratified 5-fold cross-validation is repeated across five random seeds. Within each fold, the training partition is balanced using minority oversampling and trained with class-weighted loss. The validation partition controls early stopping through macro-F1, while the held-out fold provides test predictions. Finally, fold-level predictions are aggregated to compute accuracy, macro-F1, ROC-AUC, confusion matrices, and paired statistical tests.

Missing figure file: methodology_figure.png
Alternative also missing: pipeline.png

Figure 2: End-to-end fine-tuning pipeline. The same preprocessing, splitting, training, and evaluation procedure is applied to each backbone.

The training procedure used a custom `WeightedTrainer` built on HuggingFace’s `Trainer`. The main training settings were: learning rate 2×10^{-5} , batch size 8, maximum 12 epochs, weight decay 0.05, warmup ratio 0.15, dropout 0.20, maximum sequence length 256, and FP16 training on a Kaggle T4 GPU. Class-weighted cross-entropy with label smoothing 0.05 addressed imbalance, and manual early stopping monitored validation macro-F1 with patience 4.

2.4 Technical Stack and Challenges

The implementation uses Python 3.10, PyTorch, HuggingFace Transformers, scikit-learn, and the HuggingFace Datasets library. Matplotlib and Seaborn were used for visualization. These tools were selected because they provide reliable model loading, tokenization, cross-validation utilities, metric computation, and reproducible experimentation.

The most significant engineering challenge was Kaggle’s disk-quota limit. Standard HuggingFace early stopping saves checkpoints repeatedly, which became impractical across 75 fold-runs. I addressed this by implementing a

manual early-stopping callback that tracks validation macro-F1 in memory without writing full checkpoints after every epoch. This allowed the full experiment to complete reliably under the available resource constraints.

2.5 Scope Changes from the Midterm Proposal

The final implementation diverged from the midterm proposal in three important ways. First, the original plan involved larger decoder models such as Llama 3.1 8B and DeepSeek-Coder. Because the available dataset was small, those models were less stable and more expensive to tune; the final system used smaller encoder backbones that supported controlled comparison. Second, the task changed from fine-grained multi-class taxonomy prediction to binary classical-versus-quantum classification, because several taxonomy classes had too few examples for reliable supervised learning. Third, the evaluation changed from a single 70/10/20 split to repeated 5-fold cross-validation, which produced more stable estimates and enabled matched statistical comparisons.

3 Results and Evaluation

3.1 Quantitative Results

All three backbones were evaluated under the same 25-fold protocol. Table 2 reports the mean and standard deviation of accuracy, macro-F1, and ROC-AUC. All models exceeded the original 70% accuracy target. CodeBERT achieved the highest mean accuracy and macro-F1, while GraphCodeBERT achieved the highest mean ROC-AUC.

Table 2: Cross-validation performance reported as mean $\pm$ standard deviation across 25 fold-runs per backbone.

Backbone	Accuracy	Macro-F1	ROC-AUC
RoBERTa	0.764 $\pm$ 0.061	0.754 $\pm$ 0.066	0.858 $\pm$ 0.048
CodeBERT	0.767 $\pm$ 0.057	0.763 $\pm$ 0.056	0.855 $\pm$ 0.044
GraphCodeBERT	0.758 $\pm$ 0.058	0.756 $\pm$ 0.058	0.860 $\pm$ 0.047

Paired Welch’s t-tests on matched folds showed no statistically significant differences among the three models on any metric. This means the small numerical advantage of CodeBERT on accuracy and macro-F1 should not be overinterpreted. The more defensible conclusion is that all three models performed similarly under this dataset and evaluation protocol.

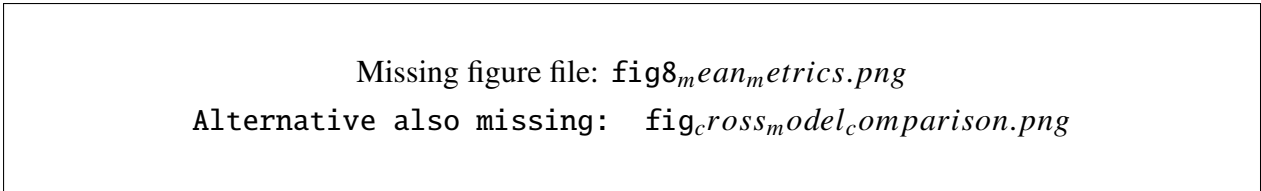


Figure 3: Cross-backbone comparison of accuracy, macro-F1, and ROC-AUC. Error bars show ± 1 standard deviation across 25 fold-runs.

The confusion matrix in Figure 4 shows that the selected classifier did not simply collapse to the majority class. Both classical and quantum bug reports were recovered with meaningful recall, indicating that class-weighted loss and minority oversampling were necessary design choices rather than cosmetic additions.

Missing figure file: `fig1_confusion_matrix.png`
Alternative also missing: `fig1_confusion_matrix_codebert.png`

Figure 4: Pooled confusion matrix for the best-performing backbone, CodeBERT, across 25 fold-runs.

Because ROC-AUC is one of the headline metrics, Figure 5 visualizes the probabilistic ranking behavior of the models. The curves are close to one another, supporting the statistical result that the three backbones are not meaningfully separated by pre-training corpus at this data scale.

Missing figure file: `fig5_roc_curve.png`
Alternative also missing: `fig_roc_overlay.png`

Figure 5: ROC curve comparison for the evaluated backbones. The similar curve shapes support the finding that backbone choice produced only minor differences.

3.2 Qualitative Observations

Manual inspection of errors revealed two recurring patterns. First, some classical infrastructure bugs were predicted as quantum when their descriptions contained Qiskit-specific imports, circuit terminology, or backend names. This suggests that the models sometimes relied on surface-level quantum vocabulary rather than deeper semantic evidence. Second, quantum-specific bugs with very short descriptions and no code snippet were more likely to be predicted as classical. In these cases, the input lacked enough contextual signal for the encoder to distinguish a quantum semantic error from a general software issue.

Overall, the system met its primary objective: it produced a reproducible binary triage classifier with accuracy above the proposed threshold and balanced performance across classes. However, the qualitative analysis shows that the classifier is better understood as an early triage tool than as a complete quantum bug diagnosis system.

4 Discussion

4.1 What Worked Well

Three design choices worked especially well. First, repeated cross-validation produced more reliable estimates than a single split would have provided. Second, class-weighted loss combined with minority oversampling prevented

majority-class collapse and improved the usefulness of macro-F1 as an evaluation metric. Third, the controlled comparison across equal-size backbones made the negative result meaningful: the study could fairly test whether code-aware pre-training mattered without confounding the comparison with model size or architecture.

4.2 What Did Not Work as Expected

The central hypothesis was that CodeBERT or GraphCodeBERT would clearly outperform RoBERTa because they were pre-trained on code-related corpora. That did not happen. Although CodeBERT had the highest mean accuracy and macro-F1, and GraphCodeBERT had the highest AUC, the differences were not statistically significant. This suggests that the classification signal in the available dataset may come more from natural-language bug descriptions than from executable code structure.

4.3 Limitations and Future Work

This project has three main limitations. First, the dataset is Qiskit-centric, so generalization to other quantum frameworks such as Cirq, Q#, or PennyLane remains untested. Second, many samples are dominated by natural-language descriptions rather than complete executable code, which limits conclusions about direct code reasoning. Third, hyperparameters were intentionally held fixed across backbones to preserve a controlled comparison, but model-specific tuning might change the relative ordering.

Future work should expand the labeled dataset through curated GitHub issue mining and controlled code perturbation. A stronger follow-up study should also implement prompt-only and retrieval-augmented baselines, as originally planned, to determine whether fine-tuning provides an advantage over inference-time prompting. Finally, a small generative model fine-tuned with LoRA or QLoRA [2, 5] could produce structured outputs containing the predicted bug type, suspected location, and a natural-language explanation.

5 Conclusion

This project fine-tuned and compared three pre-trained encoder models for binary classical-versus-quantum bug classification on a Qiskit-centric bug corpus. All three models exceeded the original 70% accuracy target, with mean accuracy in the high seventies and ROC-AUC near 0.86. The main empirical finding is that pre-training corpus did not significantly separate RoBERTa, CodeBERT, and GraphCodeBERT when architecture, parameter count, preprocessing, and training protocol were held fixed.

I personally learned that applied LLM development is not only about choosing a larger or more specialized model. Dataset quality, task framing, class balance, evaluation design, and infrastructure constraints strongly shape the final result. In early experiments, class imbalance and distribution mismatch produced misleading behavior; the final system became reliable only after the task was reframed, the evaluation was repeated across seeds, and imbalance was handled explicitly.

The broader lesson is that successful LLM applications require careful engineering and honest evaluation. A

modest encoder model, evaluated rigorously, can provide a useful domain-specific tool and a stronger research contribution than a larger model evaluated with weaker methodology.

References

- [1] Campos, J., & Souto, A. (2021). QBugs: A collection of reproducible bugs in quantum algorithms and a supporting infrastructure to enable controlled quantum software testing and debugging experiments. *arXiv:2103.16968*. <https://doi.org/10.48550/arXiv.2103.16968>
- [2] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized language models. *Advances in Neural Information Processing Systems*, 36, 10088–10115. <https://arxiv.org/abs/2305.14314>
- [3] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., & Zhou, M. (2020). CodeBERT: A pre-trained model for programming and natural languages. *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1536–1547. <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
- [4] Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S. K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., & Zhou, M. (2021). GraphCodeBERT: Pre-training code representations with data flow. *Proceedings of the 9th International Conference on Learning Representations*. <https://arxiv.org/abs/2009.08366>
- [5] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *Proceedings of the 10th International Conference on Learning Representations*. <https://arxiv.org/abs/2106.09685>
- [6] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv:1907.11692*. <https://arxiv.org/abs/1907.11692>
- [7] Zhao, P., Zhao, J., Miao, Z., & Lan, S. (2021). Bugs4Q: A benchmark of real bugs for quantum programs. *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*, 1373–1376. <https://doi.org/10.1109/ASE51524.2021.9678908>